Fundação Educacional do Município de Assis
Instituto Municipal de Ensino Superior de Assis
Campus "José Santilli Sobrinho"

GABRIELA VIANA CUNHA

DESENVOLVIMENTO DE SOFTWARE PARA GESTÃO DE CARTEIRA MÉDICA INDIVIDUAL

Assis/SP
2026

Fundação Educacional do Município de Assis
Instituto Municipal de Ensino Superior de Assis
Campus "José Santilli Sobrinho"

GABRIELA VIANA CUNHA

DESENVOLVIMENTO DE SOFTWARE PARA GESTÃO DE CARTEIRA MÉDICA INDIVIDUAL

Projeto de pesquisa apresentado ao curso de Análise de Sistemas do Instituto Municipal de Ensino Superior de Assis – IMESA e a Fundação Educacional do Município de Assis – FEMA, como requisito parcial à obtenção do Certificado de Conclusão.

Orientando(a): Gabriela Viana Cunha
Orientador(a): Professor Douglas Sanches Cunha

Assis/SP
2026

FICHA CATALOGRÁFICA

VIANA CUNHA, Gabriela.
Carteira Médica Digital / Gabriela Viana Cunha. Fundação Educacional do Município de Assis –FEMA – Assis, ano.
52.

1. Prontuário Eletrônico do Paciente. 2. Java.

CDD:
Biblioteca da FEMA

CARTEIRA MÉDICA DIGITAL

GABRIELA VIANA CUNHA

Trabalho de Conclusão de Curso apresentado ao Instituto Municipal de Ensino Superior de Assis, como requisito do Curso de Graduação, avaliado pela seguinte comissão examinadora:

Orientador: _____
Professor Douglas Sanches Cunha

Examinador: _____

Assis/SP
2026

DEDICATÓRIA

Dedico este trabalho, primeiramente, a Deus, que guiou meus caminhos e me proporcionou meios de contribuir para a melhoria da vida de outras pessoas por meio da criação deste software.

Aos meus pais, que sempre me conduziram pelo caminho do estudo e da dedicação, oferecendo apoio incondicional. Em especial, à minha mãe, pelo suporte nos momentos mais difíceis.

Ao meu noivo, por estar ao meu lado nos momentos mais importantes e por todo o apoio durante o desenvolvimento deste projeto.

Ao meu orientador e tio, por me apoiar desde os momentos que antecederam meu ingresso na graduação até as decisões mais marcantes ao longo da elaboração deste trabalho.

AGRADECIMENTOS

Agradeço à minha mãe e meu noivo, pelo apoio, incentivo e compreensão ao longo do desenvolvimento deste trabalho.

Ao meu orientador, pela essencial contribuição para o desenvolvimento deste projeto.

Aos colegas e amigos que me auxiliaram durante minha trajetória na graduação.

E as pessoas que poderei beneficiar com este trabalho, o meu sincero agradecimento.

“Quem tem um porquê para viver pode suportar quase qualquer como.”
— NIETZSCHE, Friedrich

RESUMO

O presente trabalho visa o desenvolvimento da aplicação “Carteira Médica Digital”, que tem como objetivo permitir que o paciente seja o detentor de todo seu histórico médico e dessa maneira centralizar o armazenamento e consulta de todas as informações médicas pessoais. O desenvolvimento seguirá normas e práticas da Engenharia de Software, utilizará Java, Typescript, como framework Spring Boot e Angular, contará com APIs do Google e terá todo seu ciclo de vida versionado utilizando práticas da cultura DevOps. A vantagem do sistema será de servir a qualquer usuário que tiver um browser e acesso à internet.

Palavras-chave: Gestão Médica; Prontuário Eletrônico do Paciente; Aplicativo; Java; DevOps;

ABSTRACT

This work aims to develop the application **“**Digital Medical Wallet**”**, whose objective is to allow the patient to be the holder of their entire medical history and thus centralize the storage and access to all personal medical information. The development will follow Software Engineering standards and practices, using Java and TypeScript, with the Spring Boot and Angular frameworks. It will integrate Google APIs and have its entire lifecycle versioned using DevOps culture practices. The advantage of the system is that it can serve any user who has a browser and internet access.

Keywords: Medical Management; Electronic Patient Record; Application; Java; DevOps.

LISTA DE ILUSTRAÇÕES

Figura 1: Mapa Mental da Aplicação 31

Figura 2: Estrutura Analítica do Projeto (EAP).....32

Figura 3: Diagrama de Casos de Uso do Sistema.....33

Figura 4: Diagrama de Entidade-Relacionamento 34

Figura 5: Diagrama de Classes UML 35

Figura 6: Tela de Login 42

Figura 7: Tela de Cadastro de Perfil 43

Figura 8: Tela Principal de Agendamento.....44

Figura 9: Tela de Prontuário Pessoal 45

Figura 10: Tela de Agendamento de Consulta 46

Figura 11: Tela de Consulta.....47

Figura 12: Tela de Cadastro de Exames, Receitas e Diagnóstico.....48

Figura 13: Tela de Visualização de Exames, Receitas e Diagnósticos 48

Figura 14: Tela de Visualização e Cadastro de Profissionais.....49

LISTA DE TABELAS

Tabela 1: Cronograma PMBOK Fonte: PMBOK (2021) 19

Tabela 2: Requisitos Funcionais Fonte: Elaborado pelo autor (2026) 30

Tabela 3: Requisitos Não Funcionais Fonte: Elaborado pelo autor (2026). 31

Tabela 4: UC01 – Autenticação e Gestão de Conta 37

Tabela 5: UC02 – Cadastrar Profissional de Saúde 37

Tabela 6: UC03 – Marcar Nova Consulta 38

Tabela 7: UC04 – Reagendar, Cancelar ou Arquivar Consulta 39

Tabela 8: UC05 – Gerenciar Documentos Clínicos 40

Tabela 9: UC06 – Acessar Ficha via QR Code 40

Tabela 10: UC07 – Cadastrar e Gerenciar Perfil de Paciente 41

LISTA DE ABREVIATURAS E SIGLAS

1. **API** – Interface de Programação de Aplicações
2. **CI/CD** – Integração e Entrega Contínuas
3. **CNCF** – *Cloud Native Computing Foundation*
4. **DDD** – *Domain-Driven Design*
5. **DER** – Diagrama Entidade-Relacionamento
6. **IDE** – Ambiente de Desenvolvimento Integrado
7. **JPA** – *Java Persistence API*
8. **JVM** – *Java Virtual Machine*
9. **LTS** – *Long-Term Support*
10. **ORM** – Mapeamento Objeto-Relacional
11. **SUS** – Sistema Único de Saúde
12. **TICs** – Tecnologias da Informação e Comunicação
13. **UC** – *Use Case*
14. **UML** – *Unified Modeling Language*
15. **VS Code** – *Visual Studio Code*

SUMÁRIO

1. INTRODUÇÃO 15

1.1.1. PROBLEMATIZAÇÃO 15

1.1.2. OBJETIVOS GERAIS 16

1.1.3. OBJETIVOS ESPECÍFICOS 16

1.1.4. RELEVÂNCIA OU JUSTIFICATIVA 17

1.1.5. METODOLOGIA 17

1.1.6. CRONOGRAMA 18

2. REVISÃO BIBLIOGRÁFICA DAS TECNOLOGIAS E CONCEITOS 20

2.1. TECNOLOGIAS DE DESENVOLVIMENTO 20

2.1.1. Java e Spring Boot 20

2.1.2. TypeScript e Angular 20

2.1.3. Banco de Dados H2 e Hibernate (JPA) 21

2.1.4. Flyway (Versionamento e Migração de Banco de Dados) 21

2.1.5. OAuth2 21

2.1.6. Integração com APIs de Terceiros (Google Workspace e ViaCep) 22

2.1.7. Ferramentas de Desenvolvimento 22

2.2. PRINCÍPIOS DE ARQUITETURA DE SOFTWARE 23

2.2.1. Padrão Arquitetural MVC (Model-View-Controller) 23

2.2.2. Clean Code 23

2.2.3. Clean Architeture 24

2.2.4. Domain-Driven Design (DDD) 24

2.2.5. Arquitetura Pragmática Híbrida 25

2.3. PRINCÍPIOS DEVOPS E FERRAMENTAS 25

3. MODELAGEM E DESENVOLVIMENTO 27

3.1. DESCRIÇÃO GERAL DO PROJETO 27

3.2. METODOLOGIA DE DESENVOLVIMENTO DO SOFTWARE 27

3.3. ANÁLISE DE REQUISITOS 28

3.3.1. Requisitos Funcionais 28

3.3.2. Requisitos Não Funcionais 30

3.4. MAPA MENTAL E EAP 31

3.5. DIAGRAMA DE CASOS DE USO 32

3.6. DIAGRAMA DE BANCO DE DADOS 34
3.7. DIAGRAMA DE CLASSE UML 34
3.8. ESPECIFICAÇÕES DOS CASOS DE USO 36
3.8.1. UC01 – Autenticação e Gestão de Conta 36
3.8.2. UC02 – Manter Profissional de Saúde 37
3.8.3. UC03 – Marcar Nova Consulta 38
3.8.4. UC04 – Reagendar, Cancelar ou Arquivar Consulta 38
3.8.5. UC05 - Gerenciar Documentos Clínicos (Diagnósticos, Receitas e Exames) 39
3.8.6. UC06 - Acessar Ficha via QR Code (Módulo de Emergência) 40
3.8.7. UC07 - Cadastrar e Gerenciar Perfil de Paciente (Meu Prontuário) 41
4. PROTÓTIPOS E USE CASES DESCRITIVAS 42
4.1. TELA DE LOGIN 42
4.2. TELA DE CADASTRO DE PACIENTE 43
4.3. TELA PRINCIPAL DE AGENDAMENTO 44
4.4. TELA DE PRONTUÁRIO PESSOAL E GERAÇÃO DE QR CODE 45
4.5. TELA DE AGENDAMENTO DE CONSULTA 46
4.6. TELA DE CONSULTA 47
4.7. TELA DE VISUALIZAÇÃO/CADASTRO DE EXAMES, RECEITAS E DIAGNÓSTICO 48
4.8. TELA DE VISUALIZAÇÃO E CADASTRO DE PROFISSIONAIS 49
5. REFERÊNCIAS 50

1. INTRODUÇÃO

O uso das Tecnologias da Informação e Comunicação (TICs) tem se mostrado cada vez mais relevante na área da saúde, o que contribui para a melhoria na qualidade dos serviços oferecidos e para a otimização da prestação de cuidados (CIRNE et al., 2023).

Entretanto, um desafio persistente nos serviços de saúde é o absenteísmo dos pacientes em consultas previamente agendadas, o que compromete a eficiência do sistema, gera desperdício de recursos e amplia o tempo de espera por atendimento. Segundo o Ministério da Saúde (BRASIL, 2025), o Sistema Único de Saúde (SUS) já adota estratégias para melhorar a comunicação com os usuários, como o uso do Prontuário Eletrônico, que é uma de suas recomendações prioritárias.

Com o avanço do acesso à tecnologia, torna-se possível propor soluções que não apenas facilitem o uso de ferramentas digitais, mas também incentivem e engajem os pacientes. Este trabalho tem como objetivo desenvolver uma solução tecnológica que permita ao usuário monitorar e gerenciar suas consultas, armazenar e consultar suas informações de saúde de forma organizada, afim de promover uma maior adesão aos atendimentos e reduzindo o absenteísmo.

1.1.1. PROBLEMATIZAÇÃO

A observação da jornada de pacientes, especialmente aqueles com condições crônicas, revela uma sobrecarga constante na gestão de suas próprias informações de saúde. A dependência de documentos em papel, somada à fragmentação de dados em múltiplos portais e aplicativos, cria uma barreira que dificulta a adesão ao tratamento e a autonomia. Ao reconhecer este desafio, surgiu a motivação para desenvolver um software que centralize esses dados e devolva ao paciente o controle sobre seu histórico médico.

Este sistema visa ser elaborado para atender às necessidades específicas de quem gerencia múltiplas frentes de cuidado. O objetivo é oferecer uma solução unificada que visa simplificar a organização de exames, consultas e medicamentos, afim de tornar a rotina de saúde mais segura e eficiente. Além disso, a plataforma busca fortalecer a comunicação entre paciente e profissional de saúde, fornecendo dados claros e organizados que qualificam o atendimento.

1.1.2. OBJETIVOS GERAIS

O objetivo primordial deste Trabalho de Conclusão de Curso é a criação de um software abrangente, com foco em otimizar a gestão de agendamentos de consultas por parte do paciente e todas as informações pertinentes ao prontuário médico pessoal. Esse sistema visa preencher as lacunas existentes em softwares atuais bem como incentivar ao uso de tecnologia à toda população.

1.1.3. OBJETIVOS ESPECÍFICOS

Como objetivos específicos, foram desenvolvidas diversas funcionalidades responsáveis pela construção do aplicativo. O sistema propõe o desenvolvimento de um software integrado capaz de simplificar e automatizar o processo de login e de agendamento por meio da utilização da Interface de Programação de Aplicações (API) do Google, proporcionando maior praticidade e segurança no acesso. Além disso, o aplicativo conta com uma funcionalidade que permite ao usuário realizar o agendamento de consultas de forma simples e organizada.

O sistema permite que o usuário gerencie perfis diferentes, visa também cuidadores e usuários impossibilitados de atuarem sobre o sistema.

Também foi estabelecido um sistema voltado para armazenar e gerenciar documentos médicos pessoais, que possibilita ao usuário concentrar em um único ambiente digital seus registros e informações de saúde. O software oferece funcionalidades que caracterizam o sistema como uma verdadeira “carteira médica”, permitindo que o paciente tenha acesso rápido e centralizado ao seu histórico e aos seus dados relevantes.

Outra funcionalidade importante consiste na possibilidade de o usuário registrar informações pessoais anteriores às consultas vigentes, contribuindo para a construção de um histórico médico mais completo. O sistema também fornece notificações ao usuário, com o objetivo de evitar que consultas previamente agendadas sejam esquecidas.

Além disso, o aplicativo permite que o usuário organize suas consultas de acordo com seu status, diferenciando consultas ativas daquelas já finalizadas, o que facilita o acompanhamento e a gestão do histórico de atendimentos. Por fim, o software disponibiliza uma visualização específica na qual o paciente pode inserir e disponibilizar informações

médicas importantes, possibilitando que esses dados possam ser visualizados por outras pessoas em situações em que o paciente esteja impossibilitado de apresentá-los.

Dessa forma, os objetivos específicos mencionados buscam aprimorar a eficiência, a acessibilidade e a organização dos processos envolvidos, o que torna o software uma ferramenta abrangente, prática e potencialmente útil também em situações emergenciais.

1.1.4. RELEVÂNCIA OU JUSTIFICATIVA

A gestão de dados de saúde do paciente é, hoje, um processo altamente fragmentado. Informações cruciais (como exames, laudos e históricos de consultas) encontram-se dispersas em formatos físicos, portais de clínicas diferentes e e-mails, sem uma fonte única para o principal interessado: o próprio paciente. Essa desorganização gera ineficiência manual e riscos à continuidade do tratamento, especialmente para pacientes com condições crônicas. Do ponto de vista técnico, este cenário representa uma clara oportunidade para o desenvolvimento de um sistema de gerencial de prontuário médico pessoal. O objetivo é aplicar princípios de engenharia de software para criar uma solução que resolva os desafios como centralização de dados, com um repositório digital unificando os documentos de diferentes fontes, automatização de processos ao implementar um módulo de agendamento com um serviço de notificações visando reduzir o esquecimento de consultas e procedimentos e pôr fim a melhora da experiência do usuário, com uma interface intuitiva que simplifique a complexa tarefa de gerencia a própria saúde, tornando a tecnologia uma aliada do paciente.

Portanto, o desenvolvimento deste software justifica-se pela aplicação direta da tecnologia para resolver um problema de gestão de informação real e de alto impacto, otimizando processos e entregando valor tangível ao usuário final.

1.1.5. METODOLOGIA

A metodologia empregada para o presente trabalho será realizada utilizando linguagem Java, Typescript e frameworks como Spring Boot e Angular. Seguirá etapas da Engenharia de Software para todo o ciclo de desenvolvimento do software e construção de sua documentação de acordo com as diretrizes.

O presente trabalho constará com a seguinte estrutura:

1. Introdução

   Com todas informações relevantes vistas no capítulo anterior, justificando a motivação do presente software e a organização das etapas pertinentes ao seu desenvolvimento.

2. Revisão Bibliográfica das Tecnologias e Conceitos

   Esta etapa consta com o estudo sobre todas as tecnologias a serem utilizadas durante o desenvolvimento do aplicativo. Uma breve explicação e motivação da utilização das tecnologias que foram selecionadas, frameworks, plataformas de desenvolvimento (IDEs), plataformas de testes.

3. Modelagem e Desenvolvimento

   Desenvolver o projeto do sistema usando a metodologia adotada, levantamento de requisitos, validação dos requisitos, UC completa, descrição e funcionalidades dos stakeholders (atores), diagramas de UML e DER.

4. Prototipagem do Projeto

   Descrição de todo o fluxo no qual o usuário irá interagir seguindo protótipo de alta fidelidade com sua respectiva Use Case (UC) descritiva.

1.1.6. CRONOGRAMA

De acordo com PMBOK (2021), um Cronograma é um modelo para a execução das atividades do projeto, incluindo durações, dependências e outras informações de planejamento. Para atingir os objetivos estabelecidos para este projeto construiu-se o cronograma apresentado na Tabela 1:

Atividade/Mês	Janeiro	Feveriero	Março	Abril	Maio	Junho	Julho	Agosto	Setembro
Estudo teórico de Projetos Semelhantes	x								
Reunião com o gestor para levantamento de requisitos	x								
Mapa Mental		x							
Diagrama de Caso de Uso		x							
Diagrama de Atividades		x							
Diagrama de Sequência		x							
Diagrama de Classes		x							
Prototipação			x						
Programação do Banco de Dados				x					
Programação CRUD				x	x	x			
Programação Movimentações						x	x	x	
Testes Unitários								x	
Documentação	x	x	x	x	x	x	x	x	
Defesa									x

Tabela 1: Cronograma PMBOK

Fonte: PMBOK (2021)

2. REVISÃO BIBLIOGRÁFICA DAS TECNOLOGIAS E CONCEITOS

2.1. TECNOLOGIAS DE DESENVOLVIMENTO

2.1.1. Java e Spring Boot

Java é uma linguagem de programação orientada a objetos, multiplataforma (*Write Once, Run Anywhere*) e de alto desempenho, amplamente utilizada no desenvolvimento de aplicações corporativas. A adoção de uma versão *Long Term Support* (LTS) (Java 25) garante suporte prolongado, atualizações contínuas de segurança e maior estabilidade para o sistema. Além disso, o Java 25 incorpora melhorias de desempenho, evoluções na *Java Virtual Machine* (JVM) e recursos modernos da linguagem, tornando-o uma escolha robusta e preparada para aplicações de longo prazo (ORACLE, 2025).

Para utilização com o Java 25, recomenda-se o Spring Boot 3.4 (ou superior compatível), que oferece suporte às versões mais recentes do Java e é baseado no Spring Framework 6. O Spring Boot simplifica a configuração e o desenvolvimento da aplicação, reduzindo código repetitivo (*boilerplate*) e priorizando a lógica de negócio. Entre seus principais recursos estão o servidor web embarcado, o gerenciamento automatizado de dependências e a configuração baseada em convenção, promovendo produtividade, padronização arquitetural e facilidade de manutenção (SPRING, 2025).

2.1.2. TypeScript e Angular

O TypeScript é um superconjunto do JavaScript que adiciona tipagem estática, possibilitando a detecção de erros durante o desenvolvimento e aumentando a organização, a legibilidade e a manutenibilidade do código, especialmente em aplicações de maior complexidade (MICROSOFT, 2025).

O Angular será utilizado em sua versão mais recente disponível no momento do desenvolvimento. O framework é baseado em componentização, promovendo reutilização de código, separação de responsabilidades e modularidade. A abordagem com *Standalone Components* será adotada como padrão, eliminando a obrigatoriedade de módulos (*NgModules*)

e simplificando a estrutura do projeto, tornando a configuração mais direta e organizada (ANGULAR, 2025).

2.1.3. Banco de Dados H2 e Hibernate (JPA)

O H2 Database é um banco de dados relacional *open source* e leve, escrito em Java, que pode ser executado em modo *embedded* ou como servidor. É uma excelente opção para desenvolvimento e testes devido à sua facilidade de uso e inicialização rápida (H2 DATABASE, 2025).

Hibernate é uma das implementações mais populares da *Java Persistence* API (JPA), que é uma especificação para o mapeamento objeto-relacional (ORM) em aplicações Java. O Hibernate simplifica a interação com o banco de dados, abstraindo a complexidade do SQL e permitindo que os desenvolvedores trabalhem com objetos Java, garantindo portabilidade e produtividade (HIBERNATE, 2025).

2.1.4. Flyway (Versionamento e Migração de Banco de Dados)

O Flyway é uma ferramenta de código aberto voltada para o controle de versão e migração de esquemas de banco de dados, assumindo a responsabilidade de gerenciar a evolução estrutural do banco de dados de forma confiável (REDGATE, 2025).

O uso do Flyway está apoiado em evitar perda de dados e inconsistências, ao utilizar scripts SQL explícitos e versionados. A cada inicialização, o Spring Boot em conjunto com o Flyway verifica o histórico de execuções e aplica apenas as migrações pendentes. Essa abordagem garante que a arquitetura relacional projetada — incluindo a estrita normalização e a integridade das chaves estrangeiras — seja reproduzida com exatidão em qualquer ambiente (desenvolvimento, testes ou produção), estabelecendo um processo seguro para futuras alterações no banco de dados durante o ciclo de vida do software.

2.1.5. OAuth2

O OAuth 2.0 será utilizado no projeto como o mecanismo seguro e obrigatório para delegar acesso a recursos protegidos de terceiros. Sua implementação é exigida por provedores como o Google para permitir que a aplicação manipule dados privados do usuário (como

agendamentos) sem jamais ter acesso às credenciais originais da conta (login e senha do Google). A arquitetura baseia-se na emissão e validação de *Access Tokens*, garantindo que a comunicação entre o *backend* (Spring Boot) e a API externa ocorra de forma segura, rastreável e com consentimento explícito do usuário final.

2.1.6. Integração com APIs de Terceiros (Google Workspace e ViaCep)

A integração com APIs externas é uma estratégia fundamental para expandir as capacidades do sistema sem a necessidade de reinventar arquiteturas complexas. Neste projeto, a comunicação com serviços externos ocorrerá em duas frentes principais. A primeira frente utiliza a API pública do **ViaCEP** para otimizar e padronizar o cadastro de endereços físicos, como os locais de atendimento dos profissionais de saúde. Ao informar o Código de Endereçamento Postal (CEP), o sistema consome o serviço (*Webservice*) de forma assíncrona para preencher automaticamente os dados de logradouro, bairro, localidade e unidade federativa, reduzindo significativamente a margem de erros de digitação e melhorando a experiência do usuário (VIACEP, 2026). A segunda frente integra a **Google Calendar API**, pertencente ao ecossistema *Google Workspace*. Esta integração permite a sincronização em tempo real das consultas médicas criadas na plataforma diretamente na agenda pessoal do paciente ou do cuidador, facilitando a gestão de tempo e auxiliando na redução do absenteísmo por meio da emissão de lembretes nativos dos dispositivos móveis (GOOGLE, 2026).

2.1.7. Ferramentas de Desenvolvimento

Para a implementação do código-fonte do *backend*, selecionou-se a IDE IntelliJ IDEA. A adoção dessa ferramenta justifica-se pela sua ampla aderência ao ecossistema Java e suporte nativo a frameworks como o Spring Boot. O IntelliJ disponibiliza recursos que facilitam o desenvolvimento, conferindo maior agilidade, produtividade e qualidade ao software desenvolvido (JETBRAINS 2026). Na implementação do código-fonte do frontend adotou-se o editor Virtual Studio Code (VS Code), desenvolvido pela Microsoft. Essa ferramenta destaca-se por ser leve, altamente extensível e possuir suporte nativo ao Typescript. Por meio de seu vasto ecossistema de extensões, o VS Code integra ferramentas específicas para o Angular, o que proporciona um ambiente de

desenvolvimento ergonômico e garante a manutenção de um código limpo e padronizado (MICROSOFT, 2026).

Em paralelo, para validação e teste das requisições da API RESTful desenvolvida no Backend, será utilizado a plataforma Postman. Esta ferramenta permite a construção, execução e documentação de chamadas HTTP (como GET, POST, PUT e DELETE) de maneira isolada da interface gráfica. É possível simular o comportamento do cliente web, validar o formato da transferência de informações em JSON, testar a segurança das rotas (tokens JWT) e assegurar que os contratos de integração com o Spring Boot operassem perfeitamente antes do acoplamento final com o Angular (POSTMAN, 2026);

Para a prototipagem do projeto foi escolhido a plataforma Figma, que facilita a criação de telas e exportação com alta qualidade, bem como a criação de protótipos de alta fidelidade.

2.2. PRINCÍPIOS DE ARQUITETURA DE SOFTWARE

2.2.1. Padrão Arquitetural MVC (Model-View-Controller)

O Model-View-Controller (MVC) consiste de um dos padrões arquiteturais mais consolidados na engenharia de software, projetado para separar as responsabilidades de uma aplicação em três camadas distintas interconectadas. Segundo Sommerville (2019), o modelo (Model) encapsula o estado e a lógica dos dados; a visão (View) é responsável pela renderização da interface e apresentação das informações ao usuário; e o controlador (Controller) gerencia o fluxo de requisições, processando as entradas do usuário e orquestrando as atualizações entre o modelo e a visão. A adoção do padrão MVC promove a modularidade, facilita a manutenção do código e permite o desenvolvimento paralelo, sendo a fundação estrutural primária de frameworks modernos como o Spring Boot e o Angular.

2.2.2. Clean Code

Clean Code refere-se a um conjunto de princípios e práticas para escrever código-fonte que seja fácil de ler, entender e manter. Conforme Robert C. Martin (MARTIN, 2008), um código limpo deve ser:

> "...simples e direto. Um código limpo se lê como uma prosa bem escrita. Um código limpo nunca obscurece a intenção do designer, mas sim a revela." (MARTIN, 2008).

A aplicação desses princípios resulta em sistemas mais robustos, com menos bugs e mais fáceis de estender.

2.2.3. Clean Architeture

A Clean Architecture, também proposta por Robert C. Martin (MARTIN, 2017), é uma abordagem para organizar o código de um sistema de forma que ele seja independente de frameworks, bancos de dados, interfaces de usuário e agentes externos. Ela se baseia no princípio da inversão de dependência, onde as dependências fluem para dentro, das camadas externas para as internas.

O Clean Architecture inclui benefícios que vão desde desacoplar o sistema de bibliotecas e frameworks à poder isolar os testes para regras de negócio e facilitar a manutenção do sistema. Outro ponto que podemos destacar é o fato de não depender de banco de dados, ou seja, caso esta tecnologia precise ser alterada não há acoplamento da regra de negócio com o sistema de banco de dados.

2.2.4. Domain-Driven Design (DDD)

Domain-Driven Design (DDD) é uma abordagem de desenvolvimento de software que foca na modelagem do domínio de negócio. Eric Evans (EVANS, 2003) descreve o DDD como uma forma de:

> "...colocar o foco principal do projeto no domínio central e na lógica do domínio, baseando designs complexos em um modelo do domínio." (EVANS, 2003).

O DDD ajuda a criar um modelo de software que reflete de perto a realidade do negócio, facilitando a comunicação entre especialistas de domínio e desenvolvedores e resultando em sistemas mais alinhados às necessidades do usuário.

2.2.5. Arquitetura Pragmática Híbrida

No contexto deste projeto, será abordado uma arquitetura híbrida. O sistema utilizará o padrão MVC como fundação estrutural básica, aproveitando a fluidez e a convenção que o framework Spring Boot oferece para o gerenciamento de requisições web (Controladores) e persistência de dados. No entanto, para evitar o acoplamento excessivo e modelos de classes “anêmicas” (onde a lógica de domínio “escapa” para camadas de baixo nível), os conceitos de Clean Architecture e DDD serão aplicados estrategicamente. Essa hibridização garante que a lógica de domínio central permaneça isolada e testável, unindo a agilidade de desenvolvimento do MVC com a robustez e manutenibilidade das arquiteturas limpas.

2.3. PRINCÍPIOS DEVOPS E FERRAMENTAS

O DevOps, muito além de uma cultura, representa a integração contínua entre o desenvolvimento de software (*Dev*) e as operações de tecnologia da informação (*Ops*). Seu principal objetivo é encurtar o ciclo de vida de desenvolvimento do sistema, promovendo entregas frequentes, confiáveis e com alta qualidade (MICROSOFT AZURE, 2025). Segundo a Linux Foundation (2025), a evolução dessas práticas culminou na adoção de arquiteturas nativas em nuvem (*Cloud Native*), que priorizam a automação, a conteinerização e a observabilidade para construir sistemas escaláveis e resilientes.

No contexto deste projeto, a esteira de desenvolvimento fundamenta-se no controle de versão descentralizado utilizando o Git, tendo o GitHub como plataforma central para a gestão dos repositórios e da documentação. A automação da pipeline de Integração e Entrega Contínuas (CI/CD) é orquestrada pelo GitHub Actions, responsável por garantir a validação e a construção (*build*) automatizada do código.

Para assegurar a paridade e a consistência entre os ambientes de desenvolvimento, teste e produção, adota-se a conteinerização por meio do Docker e a orquestração local com o Docker Compose. Essa abordagem permite empacotar a aplicação e todos os seus serviços dependentes em unidades isoladas, simplificando drasticamente o processo de implantação (*deployment*).

Um pilar fundamental da cultura DevOps moderna aplicada ao projeto é a observabilidade. Para monitorar a saúde da aplicação e diagnosticar anomalias, o sistema integra o OpenTelemetry — um projeto de código aberto de destaque mantido pela *Cloud Native Computing Foundation* (CNCF), organização vinculada à Linux Foundation — em conjunto com o Grafana. Essa combinação garante a coleta padronizada e a visualização em tempo real de métricas críticas e *logs* de desempenho.

A sinergia entre essas tecnologias formará a base operacional para que o sistema de gestão de prontuários médicos seja não apenas robusto e eficiente, mas também altamente manutenível, alinhando a arquitetura do software às melhores práticas contemporâneas da indústria.

3. MODELAGEM E DESENVOLVIMENTO

Este capítulo consta a modelagem do software proposto. A descrição do projeto e a metodologia a ser utilizada no desenvolvimento. Diagramas de casos de uso e especificações serão utilizados para melhor compreensão da interação do software com o usuário e APIs de terceiros, bem como a modelagem do banco de dados.

3.1. DESCRIÇÃO GERAL DO PROJETO

O projeto consiste do desenvolvimento de um software voltado para a gestão unificada do histórico médico. O sistema foi concebido para atender tanto ao uso pessoal quanto ao modo “cuidado”, permitindo que um único usuário gerencie perfis e históricos clínicos de dependentes, como familiares idosos ou crianças.

A aplicação centraliza o agendamento de consultas, armazenamento de diagnósticos, upload de exames e prescrição de receitas médicas de forma rastreável. Como diferencial de segurança e acessibilidade, o sistema gera uma Ficha de Emergência dinâmica acessível via QR Code, que disponibiliza uma rota “pública” com dados vitais (tipo sanguíneo, alergias, medicamentos de uso contínuo, patologias) para equipes de primeiros socorros, com autorização prévia. A arquitetura foi desenhada utilizando Spring Boot no backend, Angular no Frontend e H2 embutido como banco de dados no desenvolvimento, apoiada por uma esteira de DevOps para integração e monitoramento contínuos.

3.2. METODOLOGIA DE DESENVOLVIMENTO DO SOFTWARE

Para a condução da engenharia do presente software, foi adotado o modelo de ciclo de vida Iterativo e Incremental, orientado por princípios das Metodologias Ágeis. Segundo Pressman e Maxim (2021), o modelo incremental aplica sequências lineares de forma escalonada à medida que o tempo avança, dessa forma produz incrementos de software operacionais a cada ciclos, eliminando a exigência de que todo o sistema seja entregue apenas no final do projeto. O desenvolvimento foi dividido em ciclos de trabalho, as Sprints, afim de permitir entregas funcionais e revisões constantes. O escopo do sistema foi

mapeado em uma lista de requisitos e casos de uso (Product Backlog), definindo as funcionalidades de maior prioridade nas primeiras iterações.

Essa abordagem permite mitigar riscos logo nas fases iniciais, validar e testar de maneira progressiva, assegurando a robustez e a qualidade do artefato final (SCHWABER; SUTHERLAND, 2020).

3.3. ANÁLISE DE REQUISITOS

Os requisitos foram levantados com stakeholders que atuam em frente de cuidado com pacientes que estão em tratamento multidisciplinar com consultas médicas frequentes bem como praticando polifarmácia, paciente que fazem uso de uma ampla gama de medicamentos diariamente. A validação dos requisitos ocorreu com as partes interessadas bem como com outros dois Analistas de Sistemas, Guilherme Delgado e Gabriel Matheus Santos.

Os requisitos levantados foram classificados em Funcionais (o que o sistema deve fazer) e Não Funcionais (como o sistema deve se comportar em termos de qualidade, segurança e desempenho).

3.3.1. Requisitos Funcionais

Os Requisitos Funcionais (RF) descrevem as interações entre o sistema e seus atores, detalhando os serviços que o *software* deve fornecer de forma explícita e testável.

Identificador	Descrição	Prioridade
RF01	O sistema deve permitir que o Usuário cadastre sua conta, realize a autenticação e execute a exclusão lógica de sua conta principal, inativando o acesso à plataforma.	Alta
RF02	O sistema deve permitir que o Usuário autenticado cadastre, edite e inative múltiplos perfis de pacientes (para atuação como cuidador) ou crie um perfil vinculado aos seus próprios dados, integrando a busca de CEP para o preenchimento do endereço residencial.	Alta

RF03	O sistema deve permitir que o Usuário cadastre os dados de profissionais de saúde e utilize a integração com a API do ViaCEP para realizar o preenchimento automático e padronizado do endereço da clínica a partir do CEP informado.	Alta
RF04	O sistema deve permitir que o Usuário crie, liste e filtre consultas médicas, vinculando obrigatoriamente um perfil de Paciente, cadastrando um Profissional, uma data, hora de início e hora de término específicas. Adicionalmente, o sistema deve permitir o arquivamento de consultas passadas ou canceladas, ocultando-as da listagem principal sem, no entanto, apagá-las logicamente.	Alta
RF05	O sistema deve alterar o status dos agendamentos (Agendado, Reagendado, Finalizado, Cancelado) com base nas ações do Usuário, bloqueando a exclusão física do registro no banco de dados para garantir a auditoria do histórico médico.	Alta
RF06	O sistema deve permitir a sincronização opcional da criação, reagendamento e cancelamento de consultas na agenda externa do *Google Calendar*, exigindo que o Usuário conceda permissão de acesso via protocolo OAuth 2.0.	Média
RF07	O sistema deve permitir que o Usuário vincule diagnósticos a um agendamento com status "Finalizado" ou "Em Andamento", incluindo uma marcação booleana para identificar e destacar doenças crônicas.	Alta
RF08	O sistema deve permitir que o Usuário cadastre, visualize e edite registros de exames e prescrições médicas associadas a uma consulta específica para correção de eventuais erros de digitação.	Alta
RF09	O sistema deve permitir que o Usuário adicione, edite ou remova medicamentos individualmente dentro de uma receita existente, especificando a posologia e sinalizando ativamente se o fármaco é de uso contínuo.	Alta

RF10	O sistema deve permitir que o Usuário registre, visualize e exclua o histórico de alergias e reações adversas associadas ao perfil de um paciente específico.	Média
RF11	O sistema deve permitir que o Usuário realize o upload de arquivos referentes a resultados de exames ou receitas digitalizadas, processando e armazenando o documento no banco de dados em formato de string Base64.	Baixa
RF12	O sistema deve gerar automaticamente um identificador único (UUID) no momento do cadastro do paciente e fornecer um QR Code escaneável na interface do aplicativo.	Alta
RF13	O sistema deve disponibilizar uma interface pública, acessível via QR Code por socorristas (atores não autenticados), que exiba estritamente o tipo sanguíneo, alergias, diagnósticos crônicos e medicamentos de uso contínuo, validando previamente se o Usuário ativou a permissão de visibilidade da ficha.	Alta

Tabela 2: Requisitos Funcionais

Fonte: Elaborado pelo autor (2026)

3.3.2. Requisitos Não Funcionais

Os Requisitos Não Funcionais (RNF) definem as propriedades e restrições da arquitetura, incluindo regras de segurança, integrações e performance (SOMMERVILLE, 2019).

Identificador	Descrição	Prioridade
RNF01	O sistema deve armazenar dados sensíveis de autenticação (caso aplicável) utilizando algoritmos de *Hash* seguros unidirecionais.	Alta
RNF02	O sistema deve restringir o acesso a dados médicos e agendamentos exclusivamente a Usuários autenticados (validados via *Token* JWT no *backend*), com exceção da rota específica da Ficha de Emergência.	Alta

RNF03	O sistema deve consumir as APIs externas (*ViaCEP* e *Google Calendar*) de forma assíncrona, garantindo que a interface (*frontend* Angular) não seja bloqueada durante a espera da resposta do servidor externo.	Alta
RNF04	O sistema deve renderizar a rota pública de leitura do *QR Code* com alta disponibilidade e tempo de carregamento otimizado para garantir eficiência no atendimento de primeiros socorros.	Alta

Tabela 3: Requisitos Não Funcionais

Fonte: Elaborado pelo autor (2026).

3.4. MAPA MENTAL E EAP

Para facilitar a compreensão dos limites e das funcionalidades da plataforma, elaborou-se um Mapa Mental do sistema. Apresentado na Figura 1, atua como uma ferramenta de delimitação de escopo, ramificando as principais características do *software* a partir de seu núcleo.

Enquanto o Mapa Mental detalha o escopo do produto (o que será construído), a Estrutura Analítica do Projeto (EAP), Figura 2, é a ferramenta utilizada para definir o escopo do projeto (ou seja, como o trabalho será executado).

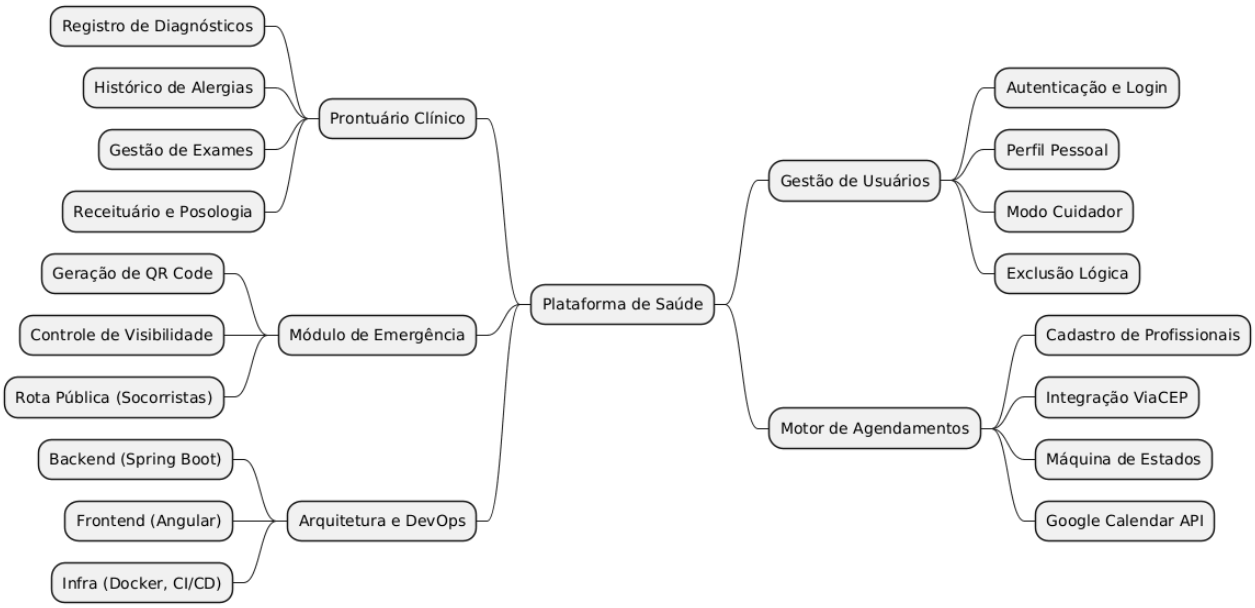

Figura 1: Mapa Mental da Aplicação

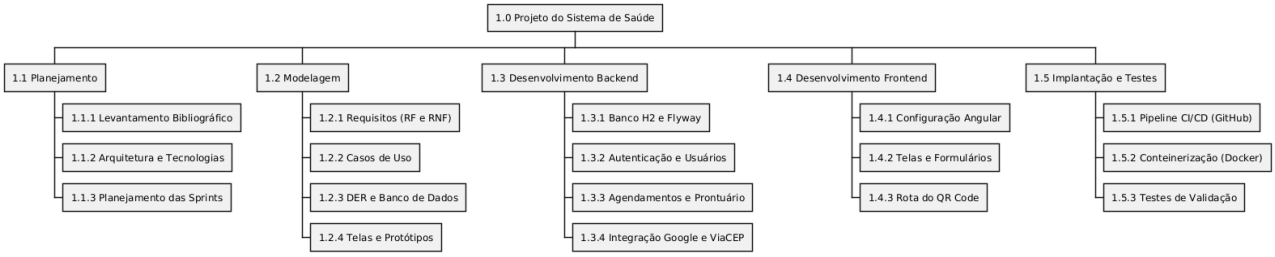

Figura 2: Estrutura Analítica do Projeto (EAP)

3.5. DIAGRAMA DE CASOS DE USO

O Diagrama de Casos de Uso constitui um dos principais artefatos comportamentais da Unified Modeling Language (UML). Segundo Sommerville (2019), é essencial para modelar as interações entre o ambiente externo (atores) e as funcionalidades que o software deve conter para que satisfaça os requisitos levantados. A Figura 3 apresenta o diagrama estruturado para o escopo do projeto.

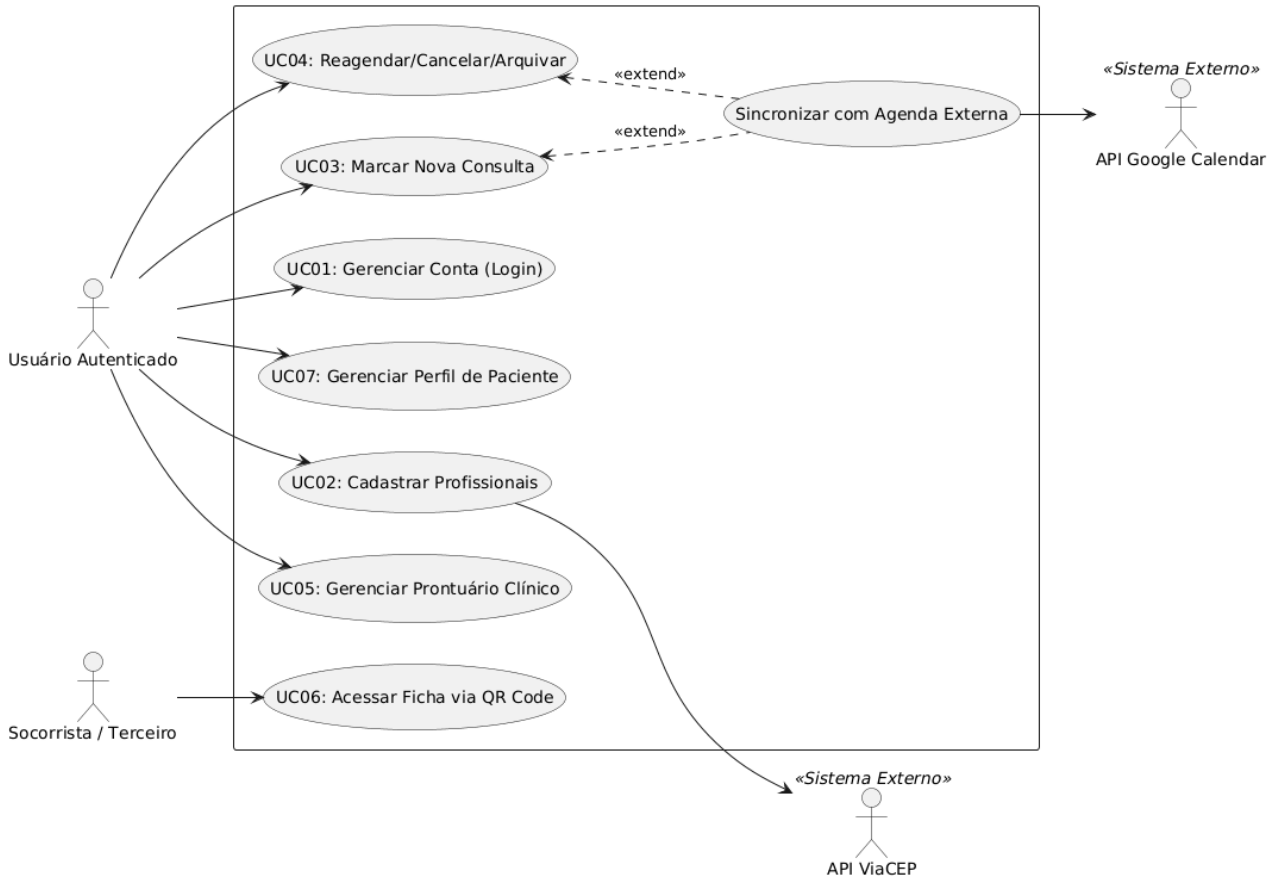

Figura 3: Diagrama de Casos de Uso do Sistema

O usuário autenticado é o ator principal, com privilégios para gerenciar a conta e os perfis dos pacientes (o que engloba a ação de administrar múltiplos dependentes no modo cuidador), cadastrar profissionais, manipular o ciclo de vida das consultas e registrar o histórico de prontuários que são as receitas e exames. Em contrapartida, o Socorrista representa um ator não autenticado cuja única interação permitida é a leitura (UC06) dos dados vitais de emergência do paciente, garantindo padrões de LGPD.

De forma adicional, o diagrama destaca o acoplamento do sistema com sistemas externos. Pode-se observar a relação de inclusão (<>) entre o cadastro de profissionais (UC02) e a API do ViaCEP, indicando que a padronização automática de endereços é uma etapa obrigatória do fluxo de negócio. Em contrapartida, a comunicação com a API do *Google Calendar* foi modelada como uma extensão (<>) dos fluxos de agendamento e cancelamento (UC03 e UC04). Essa abordagem arquitetural demonstra que a sincronização com a agenda externa é uma funcionalidade opcional, permitindo ao usuário

registrar a consulta exclusivamente no banco de dados local do sistema ou estender a ação, integrando-a ativamente ao seu calendário pessoal de forma automatizada.

3.6. DIAGRAMA DE BANCO DE DADOS

O Diagrama Entidade-Relacionamento (DER) cumpre o papel para mapear visualmente as tabelas e respectivos atributos. Segundo Elmasri e Navathe (2011), a aplicação de técnicas de modelagem estrutural aliada ao processo rigoroso de normatização é indispensável para projetar um banco de dados relacional eficiente.

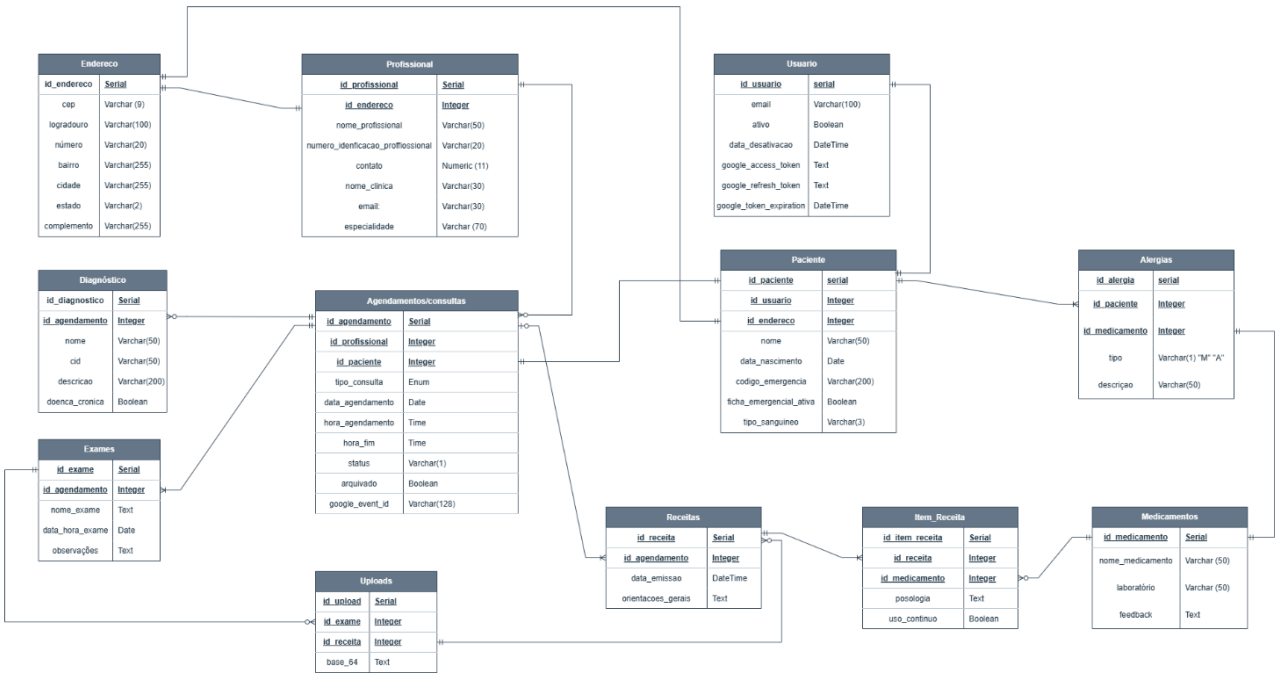

Figura 4: Diagrama de Entidade-Relacionamento

3.7. DIAGRAMA DE CLASSE UML

O Diagrama de Classes é o artefato estático responsável por mapear a estrutura de dados e as operações encapsuladas pelas entidades do domínio. Segundo Pressman e Maxim (2021), este modelo atua como a fundação arquitetural para o mapeamento objeto-relacional (ORM) na camada de persistência da aplicação.

A Figura 5 ilustra o Diagrama de Classes projetado para o sistema, refletindo a normalização do banco de dados e as regras de negócio intrínsecas ao gerenciamento de prontuários.

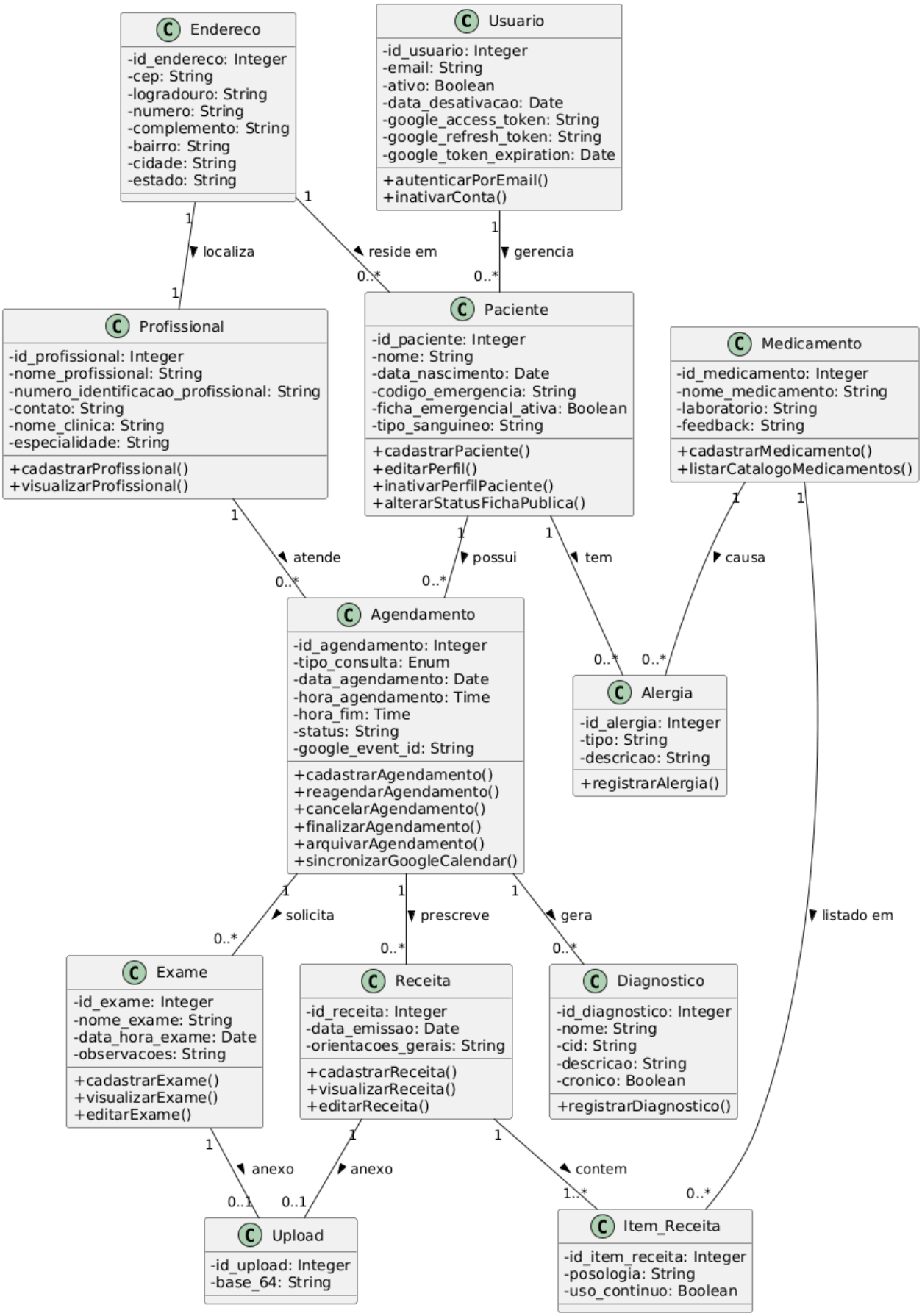

Figura 5: Diagrama de Classes UML

3.8. ESPECIFICAÇÕES DOS CASOS DE USO

As especificações dos casos de uso facilitam a visualização do fluxo do usuário diante de cada caso de uso e suas interações com o sistema.

Sua composição se dá por:

Ator Principal: usuários que utilizarão o caso de uso.

Pré-Condições: tudo que é necessário para que o usuário acesse o sistema e inicie a execução do caso de uso.

Fluxos (Principal e Alternativo): Consistem no passo a passo da aplicação, mostrando o que deve ocorrer caso o usuário siga o caminho esperado (Principal) ou caminhos secundários que também levam ao sucesso (Alternativo). Fluxo de Exceção: são as situações não previstas que podem ocorrer que alterarão a interação do usuário, saindo do fluxo principal esperado.

Fluxo de Exceção: Descreve as situações não previstas ou erros que abortam a operação, como indisponibilidade de APIs externas, dados inválidos ou violações de regras de negócio.

Fluxo de Teste: Estabelece as diretrizes e critérios de aceite práticos para que seja possível validar se o caso de uso foi implementado corretamente.

3.8.1. UC01 – Autenticação e Gestão de Conta

Ator Principal: Usuário (Titular da Conta)
Pré-condição: O usuário deve possuir um endereço de e-mail válido.
Fluxo Principal: 1. O Usuário acessa a página de login da plataforma e insere seu e-mail. 2. O Sistema gera um código de uso único (OTP) e envia para o e-mail informado. 3. O Usuário insere o código na plataforma. 4. O Sistema valida o código, autentica o usuário (gerando um Token JWT) e o redireciona para o Dashboard.

Fluxo Alternativo (Reenvio de Código): No passo 3, caso o e-mail atrase, o usuário clica em “Reenviar Código”. O sistema invalida o código anterior, gera um novo, envia para o e-mail e o fluxo retorna ao passo 3 com sucesso.
Fluxo de Exceção (Código Inválido/Expirado): Se o código inserido estiver incorreto ou expirado, o sistema bloqueia o acesso e exibe mensagem de erro.
Teste: Testar o uso do código OTP para que não possa ser reutilizado após o primeiro login bem-sucedido.

Tabela 4: UC01 – Autenticação e Gestão de Conta

3.8.2. UC02 – Manter Profissional de Saúde

Ator Principal: Usuário Autenticado.
Pré-condição: O usuário deve estar logado no sistema.
Fluxo Principal: 1. O Usuário preenche os dados básicos do profissional 2. O Usuário digital o CEP. 3. O Sistema consulta a API do ViaCEP de forma assíncrona, preenche Logradouro, Bairro, Cidade e Estado. 4. O Usuário preenche o Número e salva os dados com sucesso. 5. Também será possível alterar informações de profissionais cadastradas.
Fluxo Alternativo: No passo 3, se o ViaCEP não encontrar o CEP ou estiver fora do ar, o sistema emite um alerta e desbloqueia os campos de endereço. O usuário preenche rua e bairro manualmente e o cadastro é finalizado com sucesso.
Fluxo de Exceção: Se o usuário tentar salvar sem informar o Nome do Profissional ou a Especialidade, o sistema impede a submissão e destaca os campos em vermelho.
Teste: Testar se sem conexão com a internet o sistema libera a digitação manual do endereço.

Tabela 5: UC02 – Cadastrar Profissional de Saúde

3.8.3. UC03 – Marcar Nova Consulta

Ator Principal: Usuário Autenticado.
Pré-Condição: Existência prévia de pelo menos um Paciente e um Profissional cadastrados.
Fluxo Principal: 1. O usuário seleciona o Paciente, entra no seu perfil, seleciona “Agendar Consulta” e preenche os formulários, cadastra o profissional, data, hora de início 2. O Usuário seleciona a opção “Sincronizar com Google Calendar”. 3. O Sistema persiste a consulta no banco local, consome a API do Google via OAuth2, cria o evento na agenda externa e salva o ID gerado (google_event_id).
Fluxo Alternativo: No Passo 2, o usuário opta por não selecionar a sincronização com o Google Calendar. O sistema salva a consulta apenas no banco de dados local e finaliza a operação com sucesso, sem acionar a API externa.
Fluxo de Exceção: Se já houver uma consulta para o mesmo paciente no mesmo horário, o sistema bloqueia o cadastro e avisa sobre o choque de agendas.
Teste: Verificar se consultas criadas com o Google Calendar desmarcado geram o registro no banco local com o campo google_event_id nulo.

Tabela 6: UC03 – Marcar Nova Consulta

3.8.4. UC04 – Reagendar, Cancelar ou Arquivar Consulta

Ator Principal: Usuário Autenticado.
Pré-condição: Existência de uma consulta cadastrada na plataforma.
Fluxo Principal (Reagendamento): 1. O Usuário seleciona uma consulta "Agendada" e clica em "Reagendar".

2. Informa as novas datas/horários e salva. O Sistema altera o *status* para "Reagendado" e atualiza o evento na API do Google Calendar.
Fluxo Alternativo 1 (Cancelamento): O usuário clica em "Cancelar". O sistema solicita confirmação, altera o *status* para "Cancelado" e aciona a API externa para remover o evento da agenda do celular. **Fluxo Alternativo 2 (Arquivamento):** Na listagem de consultas (Dashboard), o Usuário seleciona uma consulta com *status* "Finalizado" ou "Cancelado" e clica em "Arquivar". O Sistema altera o atributo arquivado para *true* e oculta o card da visão principal.
Fluxo de Exceção: O sistema impede que o usuário cancele ou reagende uma consulta que já esteja com o *status* "Finalizado", desabilitando os botões na interface.
Teste 1: Executar um cancelamento via interface e consultar a base de dados via SQL para comprovar que o registro (linha) ainda existe, atestando a integridade da auditoria.

Tabela 7: UC04 – Reagendar, Cancelar ou Arquivar Consulta

3.8.5. UC05 - Gerenciar Documentos Clínicos (Diagnósticos, Receitas e Exames)

Ator Principal: Usuário Autenticado.
Pré-condição: Acesso ao detalhamento de uma consulta com *status* "Finalizado" ou "Em Andamento".
Fluxo Principal: 1. O Usuário seleciona o tipo de registro (Diagnóstico, Exame ou Receita). 2. O Usuário preenche os dados e sinaliza condições críticas (ex: marcando um diagnóstico como "Crônico" ou medicamento como "Uso Contínuo"). 3. O Usuário realiza o *upload* de um arquivo anexo (PDF ou imagem). 4. O Sistema converte o arquivo para *Base64* e persiste os dados atrelados à consulta.

Fluxo Alternativo (Registro sem Anexo): O usuário preenche os dados textuais, mas no caso de ser uma Receita, decide não fazer o *upload* de arquivo físico. O sistema salva as orientações normalmente no banco de dados.
Teste 1: Garantir que marcadores de "Doença Crônica" salvos nesta etapa reflitam de forma automática e imediata no Módulo de Emergência.

Tabela 8: UC05 – Gerenciar Documentos Clínicos

3.8.6. UC06 - Acessar Ficha via QR Code (Módulo de Emergência)

Ator Principal: Socorrista / Terceiro (Ator não autenticado).
Pré-condição: Paciente possuir a "Ficha Emergencial Ativa" e um QR Code (UUID) gerado.
Fluxo Principal: 1. O Socorrista escaneia o QR Code utilizando a câmera do dispositivo móvel. 2. O Sistema recebe o identificador, localiza o paciente e verifica a permissão de visibilidade. 3. O Sistema renderiza a Ficha Pública contendo estritamente: Tipo Sanguíneo, Alergias, Diagnósticos Crônicos e Uso Contínuo.
Fluxo Alternativo (Paciente sem histórico grave): O paciente autorizou a ficha, mas não possui comorbidades. O sistema carrega a rota com sucesso exibindo o Tipo Sanguíneo e a mensagem: "Nenhuma alergia ou condição severa relatada".
Fluxo de Exceção (Permissão Revogada): O titular desativou a visibilidade pública no aplicativo. Ao escanear, o sistema bloqueia a rota e exibe "Acesso Negado" (Conformidade com a LGPD).
Teste 1: Inspecionar a rede (Network/Payload) durante a requisição pública para garantir que os dados das outras tabelas privadas (como preços de consultas ou exames não emergenciais) não estejam sendo trafegados de forma oculta.

Tabela 9: UC06 – Acessar Ficha via QR Code

3.8.7. UC07 - Cadastrar e Gerenciar Perfil de Paciente (Meu Prontuário)

Ator Principal: Usuário Autenticado.
Pré-condição: O usuário deve estar logado no sistema.
Fluxo Principal (Novo Cadastro): 1. O Usuário acessa a tela "Meu Prontuário". 2. O Usuário preenche os dados pessoais básicos (Nome e Data de Nascimento). 3. O Usuário informa se possui alergias (selecionando "Sim" ou "Não"). Em caso afirmativo, detalha o Tipo de Alergia e a Descrição. 4. O Usuário clica no botão para "Habilitar Ficha de Emergência". 5. O Usuário clica em "Salvar". 6. O Sistema persiste os dados nas tabelas Paciente e Alergia. 7. O Sistema gera automaticamente um identificador único (UUID), cria o QR Code e o exibe na interface para o usuário.
Fluxo Alternativo (Edição de Perfil): O usuário acessa um perfil já existente. A tela é carregada com os dados e o QR Code já visíveis. O usuário altera alguma descrição e clica em salvar. O sistema apenas realiza o *Update* no banco de dados, mantendo o mesmo QR Code (UUID) intacto.
Fluxo de Exceção (Desabilitar Ficha Pública): O usuário desmarca a opção "Habilitar Ficha de Emergência" e salva. O sistema altera a *flag* `ficha_emergencial_ativa` para *false* no banco de dados. O QR Code gerado anteriormente passa a retornar "Acesso Negado" caso seja escaneado por um socorrista.
Teste 1: Verificar se ao desabilitar a ficha de emergência, o escaneamento do QR Code bloqueia imediatamente o acesso aos dados.

Tabela 10: UC07 – Cadastrar e Gerenciar Perfil de Paciente

4. PROTÓTIPOS E USE CASES DESCRITIVAS

4.1. TELA DE LOGIN

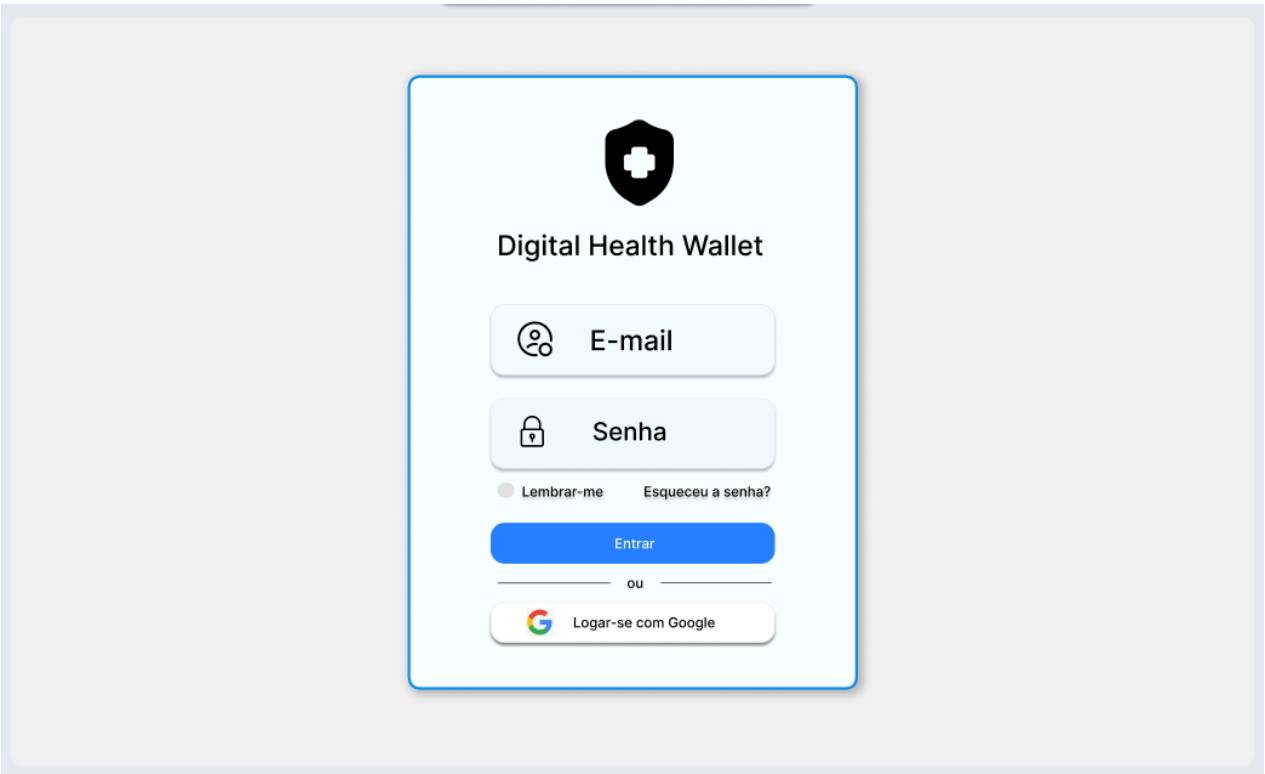

Figura 6: Tela de Login

Na figura 6, é possível ver a tela de login do sistema, onde o usuário poderá se autenticar recebendo um código via e-mail ou diretamente pelo Google.

4.2. TELA DE CADASTRO DE PACIENTE

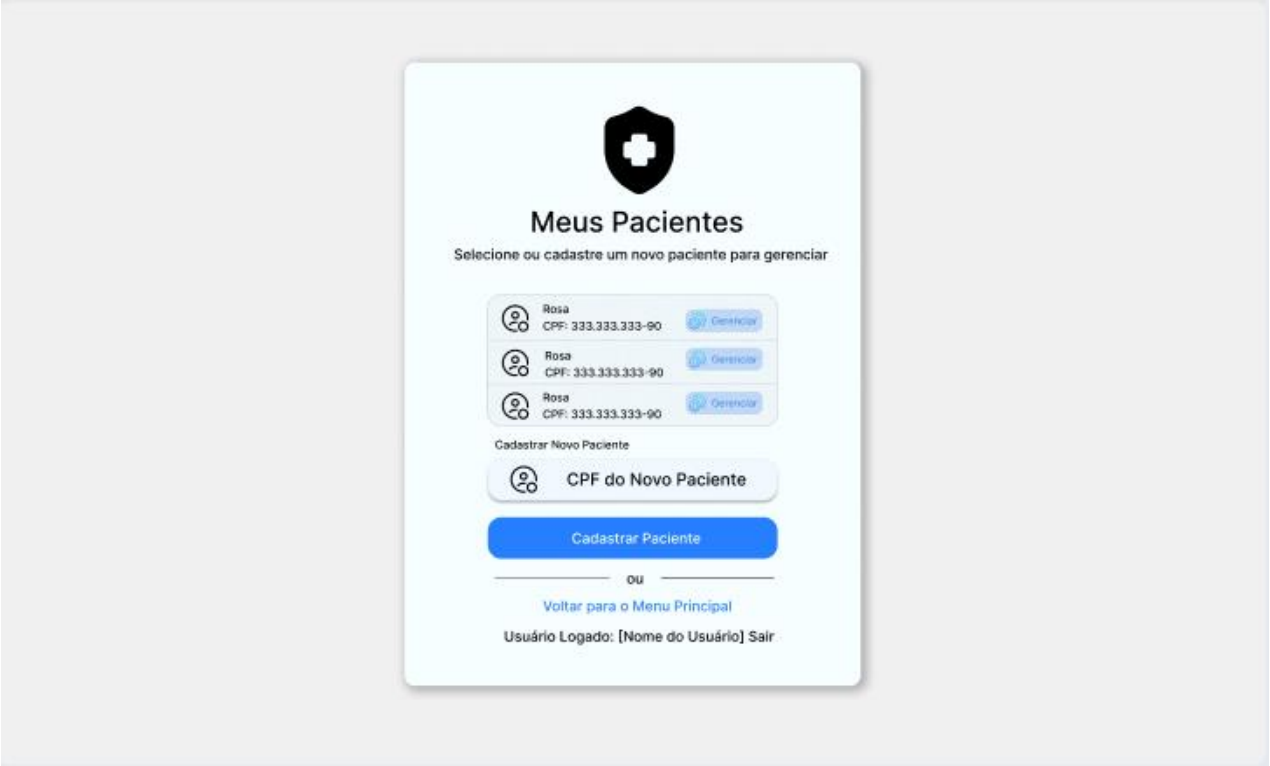

Figura 7: Tela de Cadastro de Perfil

Na figura 7, é possível visualizar a tela de cadastro de perfil. Nesta tela o usuário pode cadastrar a sai mesmo bem como ser mantenedor de mais perfis.

4.3. TELA PRINCIPAL DE AGENDAMENTO

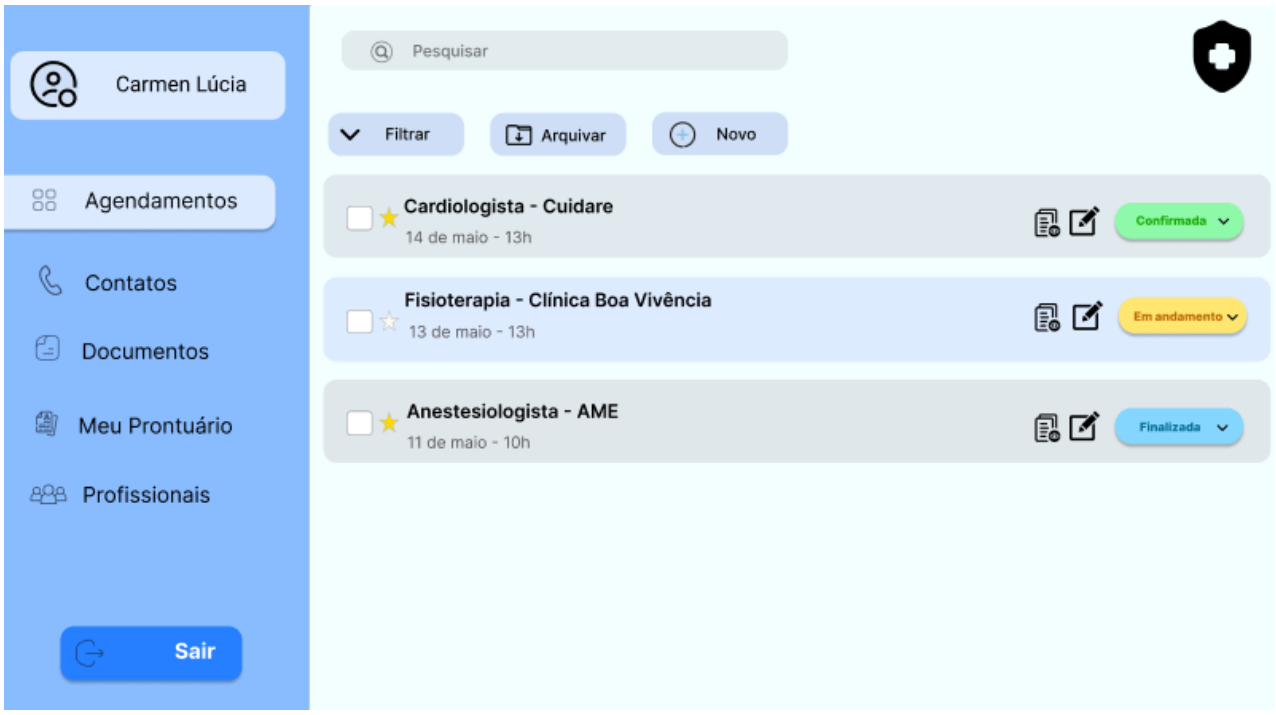

Figura 8: Tela Principal de Agendamento

Na figura 8, é possível visualizar a tela principal. Nela é possível gerenciar agendamento, cadastrar uma nova consulta, bem como navegar por outras telas através da sidebar disposta à esquerda.

4.4. TELA DE PRONTUÁRIO PESSOAL E GERAÇÃO DE QR CODE

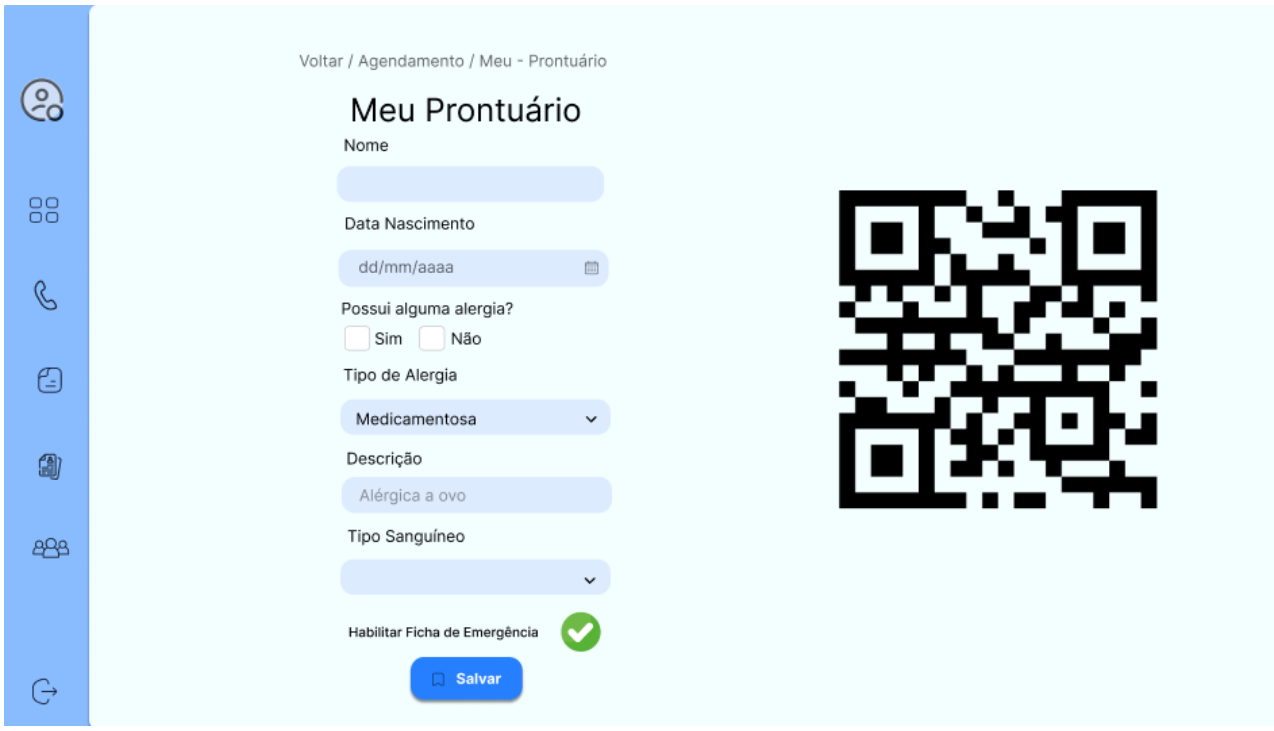

Figura 9: Tela de Prontuário Pessoal

Na figura 9, temos a tela de cadastro de dados médicos pessoais pregressos avulsos a qualquer atendimento inicial. Também é possível a geração do QR Code para a Ficha de Emergência ao habilitar a opção.

4.5. TELA DE AGENDAMENTO DE CONSULTA

Figura 10: Tela de Agendamento de Consulta

Na figura 10, a tela de consulta possibilita o cadastro/agendamento de uma consulta propriamente dita juntamente com o cadastro do profissional vinculado aquele atendimento. É possível a sincronização com o Google Agenda que irá trafegar os dados em ambas aplicações. Facilitando a comunicação com outros dispositivos. Nota, o sidebar muito similar ao da primeira página é intencional, para melhorar a experiência do usuário e seu aprendizado no sistema, uma vez que se torna repetitivo os ícones e cores nos quais ele interage.

4.6. TELA DE CONSULTA

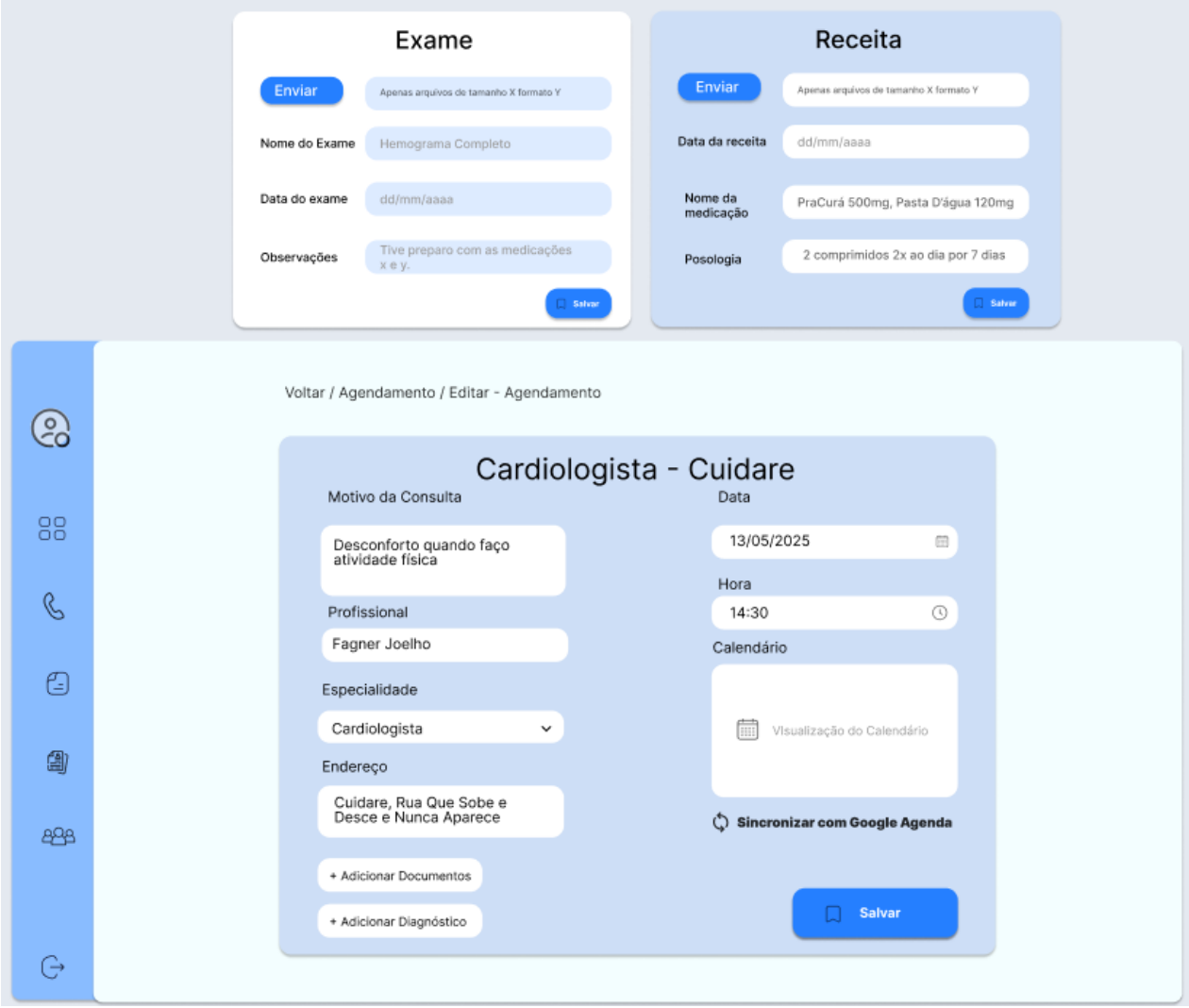

Figura 11: Tela de Consulta

Na figura 11, a tela de consulta ficará pronta e será apresentada ao usuário assim que ele finalizar/salvar o agendamento de uma consulta. Assim ele poderá editar e adicionar exames, receituários, de forma a compor seu histórico médico. Ou seja, poderá organizar seus documentos médicos de acordo com as datas nas quais foram realizados ou expeditos.

4.7. TELA DE VISUALIZAÇÃO/CADASTRO DE EXAMES, RECEITAS E DIAGNÓSTICO

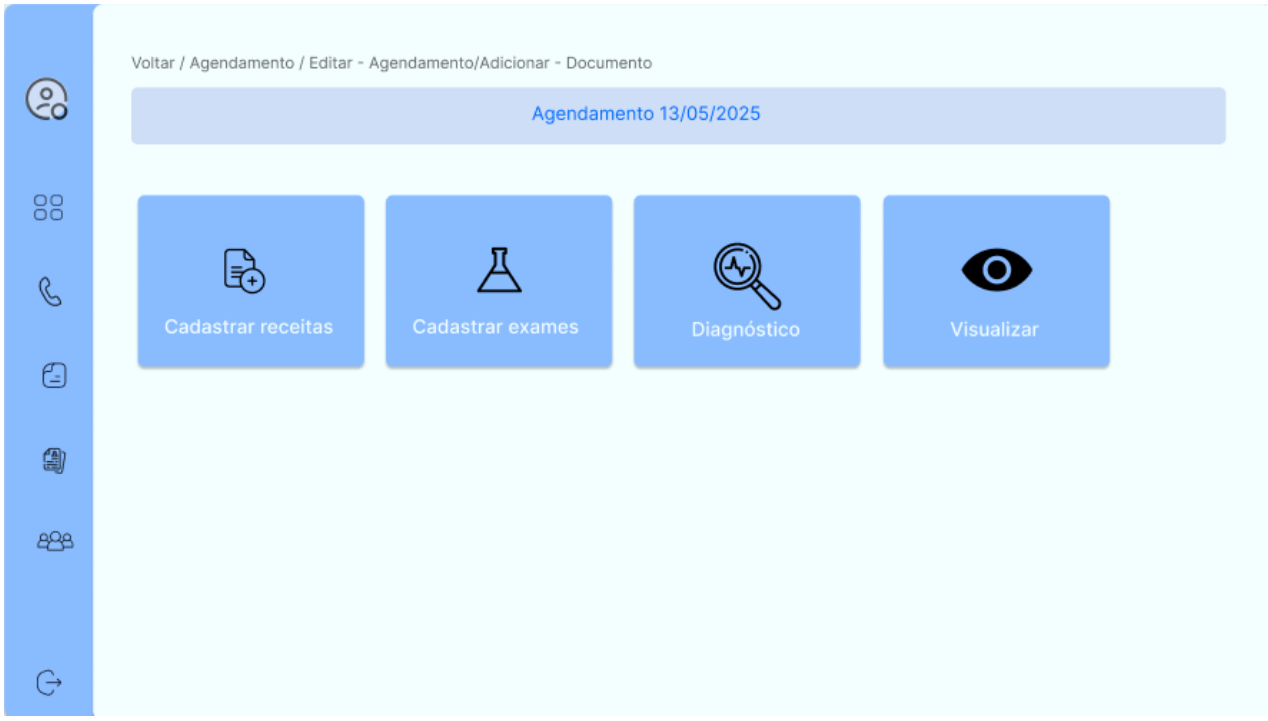

Figura 12: Tela de Cadastro de Exames, Receitas e Diagnóstico

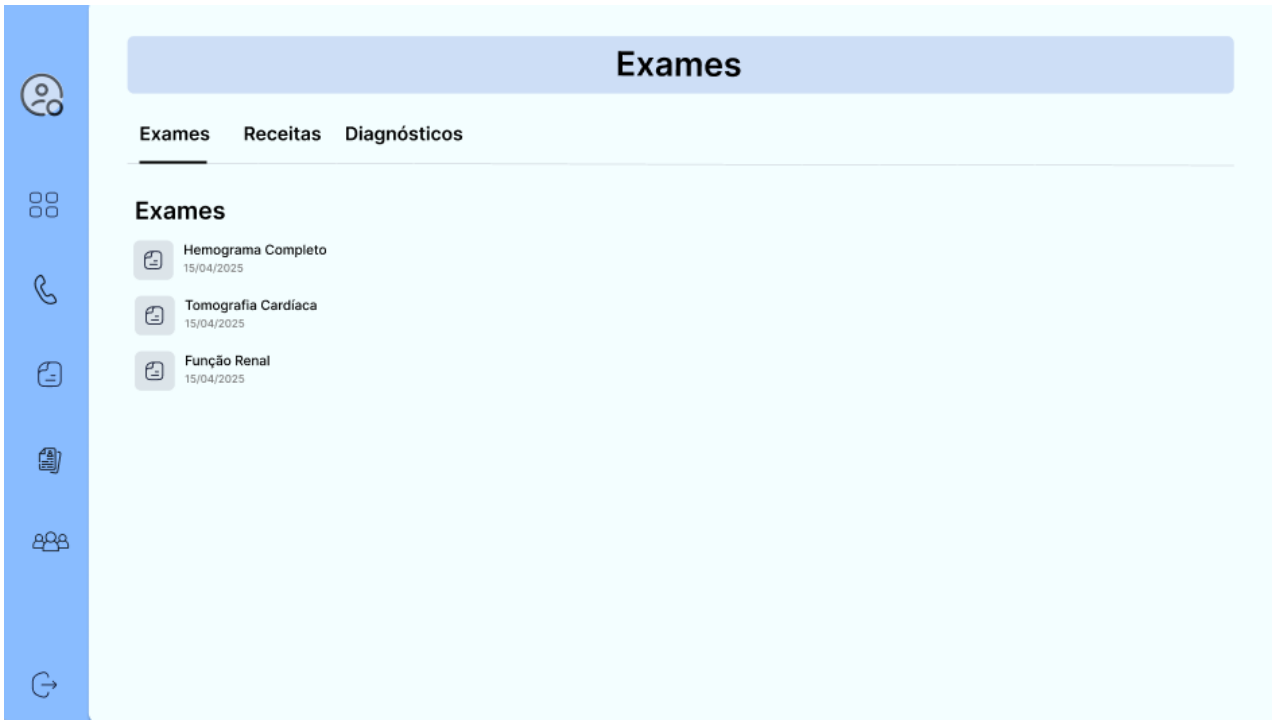

Figura 13: Tela de Visualização de Exames, Receitas e Diagnósticos

Na figura 12, é possível que o usuário cadastre exames, receitas e diagnósticos vinculados aquele agendamento o que facilitará a visualização posterior. Ele poderá visualizar esses documentos em outra tela sem o filtro de agendamento.

Na figura 13, a tela de visualização dos documentos, ordenados por nome e data de realização.

4.8. TELA DE VISUALIZAÇÃO E CADASTRO DE PROFISSIONAIS

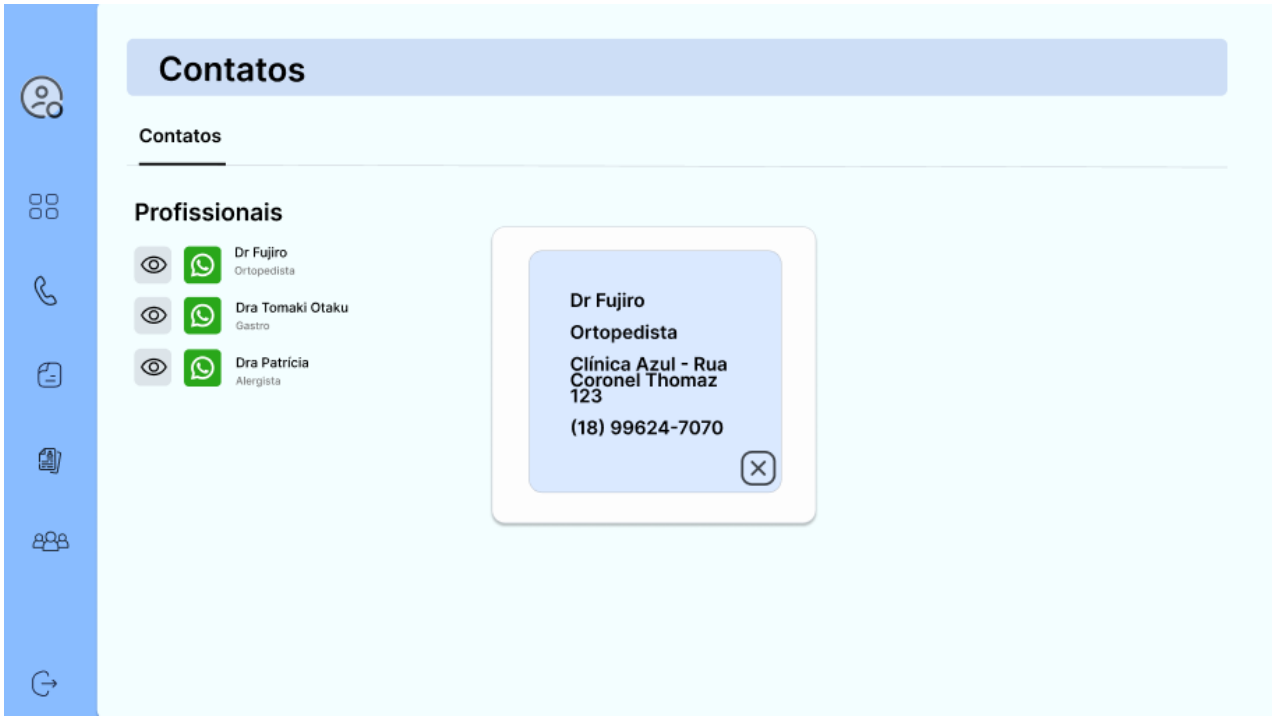

Figura 14: Tela de Visualização e Cadastro de Profissionais

Na figura 14, a tela de contatos possibilita a centralização de endereços e contatos de profissionais. O cadastro é feito via agendamento e a separação na hora da visualização facilita para que o usuário não precise entrar no agendamento para visualizar um contato profissional.

5. REFERÊNCIAS

ANGULAR. **Overview**. Disponível em: https://angular.dev/overview. Acesso em: 17 jun. 2025.

BRASIL. Ministério da Saúde. **Absenteísmo no Sistema Único de Saúde (SUS)**. Wiki da Regulação em Saúde. Disponível em: https://wiki.saude.gov.br/regulacao/index.php/Absente%C3%ADsmo_no_Sistema_%C3%9Anico_de_Sa%C3%BAde_(SUS. Acesso em: 17 jun. 2025.

CIRNE, M. R.; CYRINO, C. M. S.; MARTINS, M. R.; FRANZINI, O. R. da S.; CONSORTI, E. M.; BICUDO, T. B.; JULIANI, C. M. C. M. **Tecnologia de informação e comunicação na saúde: revisão integrativa**. Revista Recien - Revista Científica de Enfermagem, [S. l.], v. 13, n. 41, p. 890–904, 2023. DOI: 10.24276/rrecien2023.13.41.890-904. Disponível em: https://recien.com.br/index.php/Recien/article/view/807. Acesso em: 17 jun. 2025.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

FIGMA. Criar protótipos no Figma. Disponível em: https://help.figma.com/hc/pt-br/articles/360040314193-Guia-de-prototipagem-no-Figma. Acesso em: 30 mar. 2026.

GOOGLE. **Create events: Google Calendar API**. Google Workspace for Developers. Disponível em: https://developers.google.com/workspace/calendar/api/guides/create-events?hl=pt-br. Acesso em: 12 mar. 2026.

HARDT, D. (Ed.). **The OAuth 2.0 Authorization Framework**. RFC 6749, Internet Engineering Task Force (IETF), out. 2012. Disponível em: https://datatracker.ietf.org/doc/html/rfc6749. Acesso em: 11 mar. 2026.

JETBRAINS. **IntelliJ IDEA: Capable and Ergonomic Java IDE**. Praga: JetBrains s.r.o., 2026. Disponível em: https://www.jetbrains.com/idea/. Acesso em: 13 mar. 2026.

LINUX FOUNDATION. **Cloud Native Computing Foundation (CNCF) - Overview and Projects**. San Francisco: The Linux Foundation. Disponível em: https://www.cncf.io/. Acesso em: 11 mar. 2026.

MICROSOFT AZURE. **What is DevOps**. Disponível em: https://azure.microsoft.com/pt-br/resources/cloud-computing-dictionary/what-is-devops. Acesso em: 17 jun. 2025.

MICROSOFT. **Visual Studio Code: Code editing. Redefined**. Redmond: Microsoft Corporation, 2026. Disponível em: https://code.visualstudio.com/. Acesso em: 13 mar. 2026.

ORACLE. **Documentação Java SE 21**. Disponível em: https://docs.oracle.com/en/java/javase/21/. Acesso em: 17 jun. 2025.

POSTMAN. **Postman API Platform**. São Francisco: Postman, Inc., 2026. Disponível em: https://www.postman.com/. Acesso em: 13 mar. 2026.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.

REDGATE. **Flyway: Database Migrations Made Easy**. Flyway Documentation. Disponível em: https://flywaydb.org/. Acesso em: 11 mar. 2026.

SCHWABER, K.; SUTHERLAND, J. **O Guia do Scrum: O Guia Definitivo para o Scrum: As Regras do Jogo**. Scrum.org, 2020. Disponível em: https://scrumguides.org/. Acesso em: 11 mar. 2026.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019.

VIACEP. **Webservice CEP e IBGE gratuito**. Disponível em: https://viacep.com.br/. Acesso em: 11 mar. 2026.